ULL – CMPS 422 – HW1 REPORT

Braden Begnaud

This report is broken into section corresponding to the questions covered

Question 1 –

My solution for this portion was relatively straightforward and took me little time, so I decided to visit the topic presented in class, comparing my runs to an LLM generated algorithm designed from prompts and the rubric. I have full access to Google Gemini 3 and Claude Sonnet 4.6, so I used those two for my demonstrations. I've used both pretty extensively, but I'm more familiar with prompting Gemini, though I provided the same prompt and context for each. Each LLM got 3 separate tries, the first being just asking the LLM to Simply calculate it from the values rather than code an example. For every code run, the assumption was always x < t = noise, x > t = signal. This they did well, the results shown below:

Gemini Pro 3	Claude Sonnet 4.6
Build time: 0 Run time: 2 minutes Number Correct: 13 X-Value: 0.51	Build time: 0 seconds Run Time: 35 seconds Number correct: 13 X-Value: 0.68 Note: Decided the data isn't linearly separable but still gave an answer

The fields given are described as: build time being the time it took to produce code, run time is the active run time of the program in its active state, number correct and x value being the classifier results, x value being the decided splitting point and number correct being the correctly classified data points at that chosen x. As we can see, Claude was much better at the reasoning behind this task. But the assignment was to produce code,

so I tested that as well. In the first prompt, I gave the full rubric for question 1 and provided an explanation of the CSV format expected. Here are the results:

Attempt 1

Gemini Pro 3	Claude Sonnet 4.6
Build time: 35 seconds Run time: 3000ms No results Fail reason: confused over the CSV document, fails on a String to float conversion	Prompt Claude Sonnet 4.6: Build time: 45 seconds Didn’t compile Fail, didn't extract X properly, TP, TN, FP and FN as well as a graph though, fails on string to float conversion

While they produced the code really fast, they both had the same issue of converting the incoming data from a string to a float. Very similar code, but Sonnet went much further with calculations of TP, TN, FP and FN. Neither produced the asked forF-1 measure, and Gemini only gave best accuracy. The timing was also broken for Gemini, and I had to remove and re-add the timing code, which was still better than Sonnet missing it altogether.

Attempt 2

Attempt two was much worse for Gemini. Since Sonnet was close, I gave Gemini another chance with a more detailed prompt. It tried to classify ‘noise’ as an int of 0, and assumed there were no headers. This is odd since I gave even more detail on how to go about the problem and the csv, including the classifier values and the headers. It gets mixed up, and assumed that there were three rows, backtracked and decided to take the first column off to remove the header, as I instructed. I find this very interesting, considering it had the right answer, produced an incorrect answer and assumed it to be true, then did the correct path anyway. In the end, it still fails as noise is not an integer. It took 40 seconds to reason and produce the code.

Attempt 3

After two failed attempts, I decided to simply give the *exact* dataset and still keep the detailed explanation. This time, I got functional code:

Gemini Pro 3	Claude Sonnet 4.6	Me
Build time: 40 seconds Run time: 1000ms Number Correct: 13 X-Value: 0.52 Note: failed to properly time	Prompt Claude Sonnet 4.6 with CSV Explicitly: Success, lots of prints Build time: 64 seconds Run time: 3000ms with lots of prints Number correct: 13 X-Value: 1.22 Refactored without prints: Success Build time: 40 seconds Run time : 2000ms Number Correct: 13 X-Value: 0.52 Note: Refactoring changed the values, but did save ~1 second. Claude was looping for some prints	Build time: 1.5 hours (including learning) Run time: 4ms Number Correct: 13 X-Value: 0.5

For my time, it was roughly split 50/50 between learning concepts, new libraries and trying new code. Overall, the prompting and cleaning took 20 minutes or so of time, and reasonably could be cut down into one or two prompts on a familiar model. We also see that Sonnet actually got two entries for this section because it had dozens of print statements, some nested in loops, so many that removing them actually cut the run time by 1/3. What I find most interesting, however, is how mine runs so significantly faster, up to 250x than Gemini at 1000ms. My code runs on a for loop nesting two whiles and another for, not the cleanest. Claude and Gemini did it with a while in a for and a for nested within a for, respectively.

Retrospective

Overall, the LLM models needed a good bit more hand holding than I was led to believe, but still did really good work from the problem statement. I'd say without background knowledge, it would have taken time to narrow down what I wanted, but with what I learned, this project could be done and satisfactory in 5-10 minutes of prompting and review. Their solutions did run long, realistically not mattering, but still interesting that they all took several seconds to run while mine didn't. With how many sources there are for basic machine learning with python, I'm surprised it was actually a bit of an issue for them to tackle. It was helpful to have the exact input data expected, but it seems more difficult to provide them the exact format they need to follow, so maybe a diagram could help in this regard. Interestingly, providing more context and the exact CSV didn't really increase thinking time, which shows the power of the models training. Since the reasoning is the hardest part, it takes up significantly more time than actually producing what is essentially an output to a function/several functions.

Here is a set of outputs from my algorithm as it learns:

Max Correct: 11 Accuracy: 55.0% Index: 19 X-Value: 0.18 Sensitivity 1.0 F-Score: 0.6896551724137931	Max Correct: 12 Accuracy: 60.0% Index: 27 X-Value: 0.26 Sensitivity 1.0 F-Score: 0.7142857142857143	Max Correct: 13 Accuracy: 65.0% Index: 51 X-Value: 0.5 Sensitivity 0.8 F-Score: 0.6956521739130435

When checking the run yourself, the first iterations show up as -0.01 until a better value is selected.

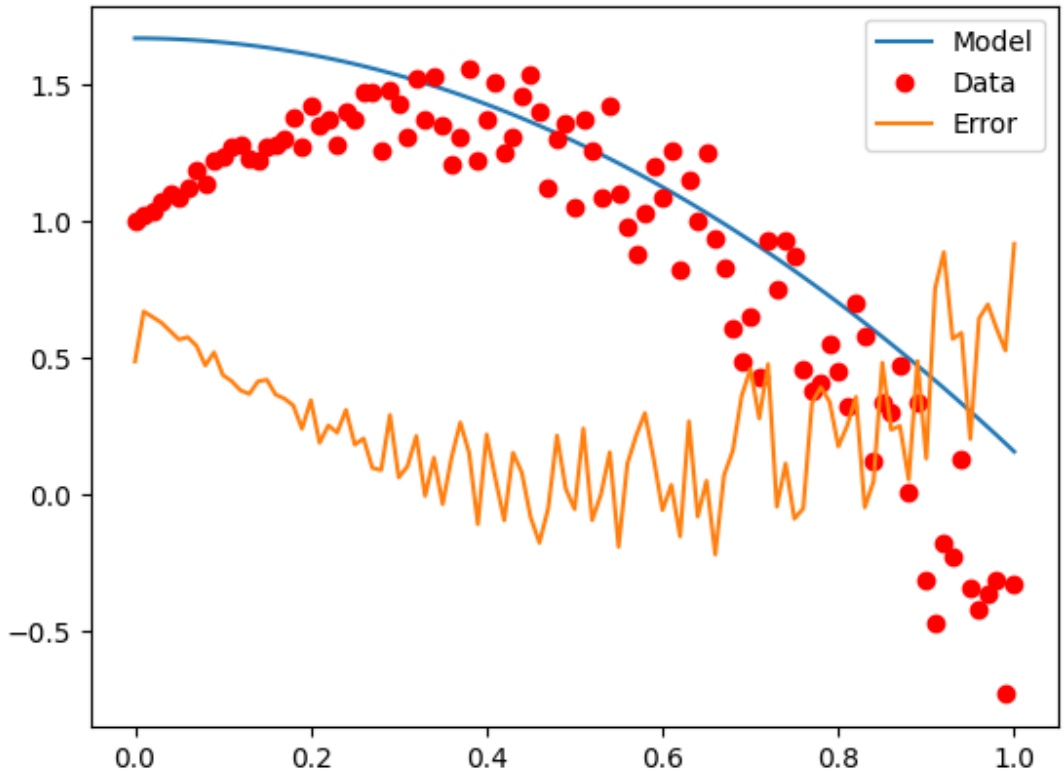
Question 2 –

My solution for this part of the homework simply involves traveling through every t value between 0.01 and 1.90 to select the classification of an x value. My algorithm found the least loss as X = 0.52 with a loss of 0.35, or 7 wrong. It loops through the dataset classifying using t as the splitting point, x < t = noise and x > t = signal. Looking at other loss functions (specifically a chart from https://www.datacamp.com/tutorial/loss-function-in-machine-learning) I see others such as Mean Square Error (which they call L2 loss and MAE L1 Loss) for regression and some other ones for classifying specifically. The rubric said to use MAE so I did, but this website says it isn't fit for a regression, which does make sense considering it's numerical but I got around this by classifying noise = 0 and signal = 1.

Question 3 –

For the weights, I used the w = (X^tX)^-1 * X^tY function, and I did this through using NumPy to give me access to linear algebra functions. It was a simple check after to check the error from the output w and b from the weight function, and to plot the actual data against my estimated. I then loop through the model's values, calculate the MAE, and plot that too, giving a nice visual. Since we are told not to iterate over degrees and check error, I simply looked at the plot and decided to use x^2.

Figure 1



As shown in the graph, the model doesn't do too good of a job predicting data at the beginning or the end of the data set, but does a nice job in the middle, quite often hitting very close to 0 error. The actual code output just shows the MAE, which is shown as 28.462 for the shown model function.

Question 4 –

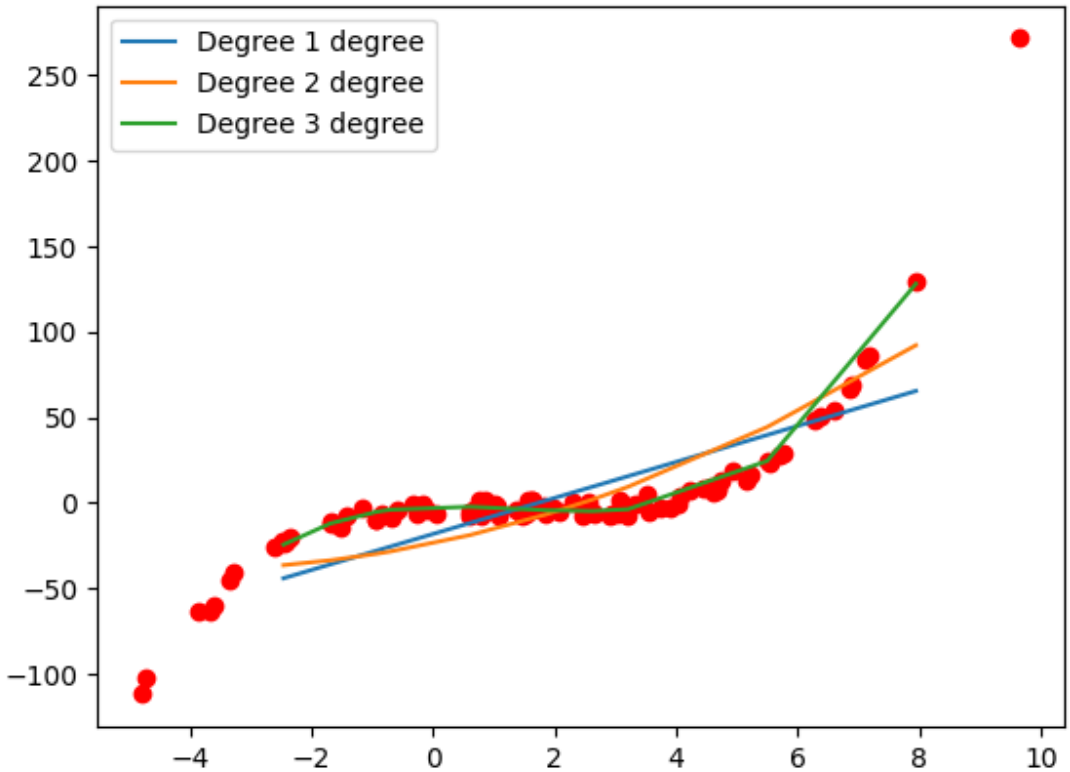
For question 4, there is no associated code. Looking at the dataset, it looks like it's cleanly separated, which is wonderful. First, I would find a dataset of positive and negative word associations. This will help me actually identify how strongly positive or negative a review is. I'm sure with how large and how cleanly separated the dataset is it would be possible to train without this, but running it against a word dataset could help when the model needs to

make predictions on not so highly polarized data. The algorithm would scrape through the dataset and look for words stored within our word dataset. This is computationally taxing, however, so the dataset needs to include the more popular words and focus less on rarer words. If possible, I could like to also preprocess the text coming it. Without benchmarks, I assume that checking over 30 string inputs and removing common words is much faster than comparing each string to dozens or hundreds of words in two or more word datasets. Since we are trying to simply classify the data, but I don't want to be too restrictive, I would design the algorithm to give each input a score. The score would range from -1 to 1 and would have ranges of, say, -1 to -0.5 are strongly negative, -0.49-.01 are weakly negative, 0 for neutral, etc. Negative values would subtract from the score, positives add.

Question 5-

For this question, I got to look into scikit which was awesome. Once I finished my look around at the libraries we were allowed to use, I built my algorithm and plots. I had issues with the plotting of my predicted values, but I managed to find a way to sort them and give a nice model output. Here is my output in text and graphical form:

Figure 2



Degree 1	Degree 2	Degree 3
MSE: 702.603 Bias: 662.439 Variance: 40.164	MSE: 513.269 Bias: 404.335 Variance: 108.934	MSE: 6.970 Bias: 6.314: Variance: 0.656

We can see here that degree 3 has a massive lead over the others, having by far the lowest mean squared error, bias and variance! For the modeling, I created a function that sets up our polynomial and fits it to the data, then creating a prediction set. Using the training and testing data, I use bias_variance_decomp from scikit to find MSE, bias and variance. From here, I resolve my issue with erratic data by sorting and flattening it (using NumPy) into a 1D array

which I can easily plot. Since the MSE, bias and variance were so high for the first two degrees, I decided not to plot them on the chart so we could actually see the fit of the third degree.

Overall, this project was actually a lot of fun to put together. I've always been a huge machine learning nerd, so this project getting down into the weeds and manually coding some of the algorithms was really exciting to me.